

```
!apt-get install -y ffmpeg
!pip install -q pydub librosa soundfile scipy matplotlib numpy
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ffmpeg is already the newest version (7:4.4.2-0ubuntu0.22.04.1).
0 upgraded, 0 newly installed, 0 to remove and 41 not upgraded.
```

```
import requests
from pydub import AudioSegment

# Using a new, publicly accessible MP3 URL
audio_url = "https://www.learningcontainer.com/wp-content/uploads/2020/02/Kalimba.mp3"

# Download paths
mp3_path = "song.mp3"
wav_path = "song.wav"

# Download the MP3 file
r = requests.get(audio_url, allow_redirects=True)
r.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)
with open(mp3_path, "wb") as f:
    f.write(r.content)

print(f"Downloaded MP3 to: {mp3_path}")

# Convert MP3 → WAV using FFmpeg (via pydub)
audio = AudioSegment.from_mp3(mp3_path)
audio.export(wav_path, format="wav")

print("Downloaded & converted to WAV:", wav_path)
```

```
Downloaded MP3 to: song.mp3
Downloaded & converted to WAV: song.wav
```

```
import librosa
from IPython.display import Audio, display

x, sr = librosa.load(wav_path, sr=22050, mono=True)

print("Audio Loaded Successfully!")
display(Audio(x, rate=sr))
```

```
Audio Loaded Successfully!
```

```
0:00 / 5:48
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, lfilter
import librosa.display

def butter_lowpass_filter(data, cutoff, sr, order=6):
    nyq = sr / 2
    normal_cutoff = cutoff / nyq
    b, a = butter(order, normal_cutoff, btype="low")
    y = lfilter(b, a, data)
    return y

def plot_spectrogram(sig, title):
    S = librosa.stft(sig)
    S_db = librosa.amplitude_to_db(np.abs(S) + 1e-10, ref=np.max)
    plt.figure(figsize=(10,3))
    librosa.display.specshow(S_db, sr=sr, x_axis="time", y_axis="hz")
    plt.title(title)
    plt.colorbar()
    plt.show()
```

```
cutoffs = [500, 2000, 5000]

for cutoff in cutoffs:
    print("\n=====")
    print(f" LOW-PASS FILTER CUTOFF = {cutoff} Hz")
    print("=====")

    y = butter_lowpass_filter(x, cutoff, sr)
```

```
display(Audio(y, rate=sr))
```

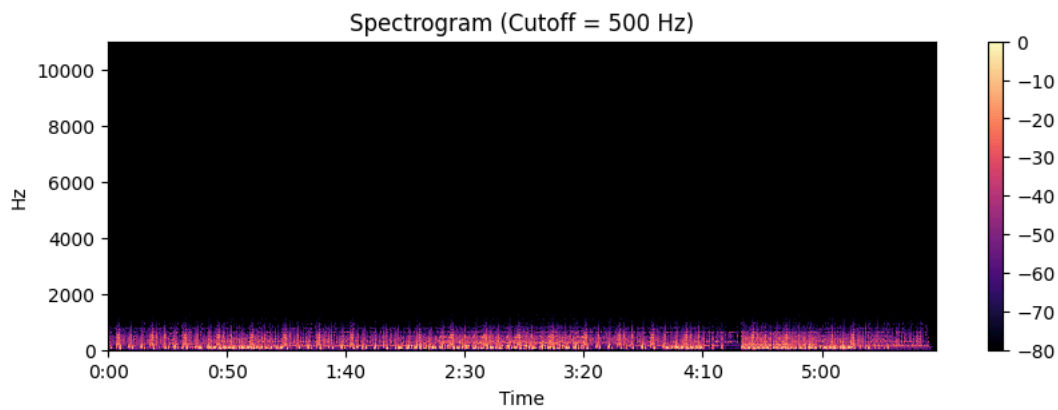
```
plot_spectrogram(y, f"Spectrogram (Cutoff = {cutoff} Hz)")
```

```
=====
```

LOW-PASS FILTER CUTOFF = 500 Hz

```
=====
```

0:00 / 5:48

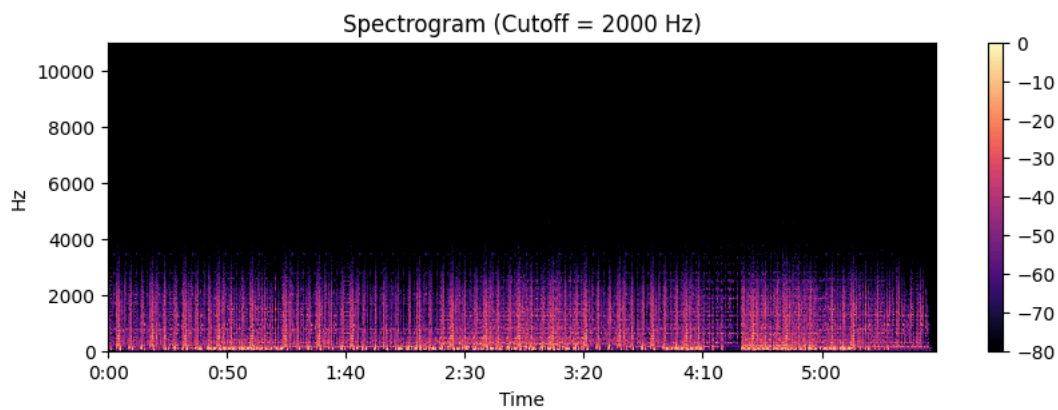


```
=====
```

LOW-PASS FILTER CUTOFF = 2000 Hz

```
=====
```

0:00 / 5:48

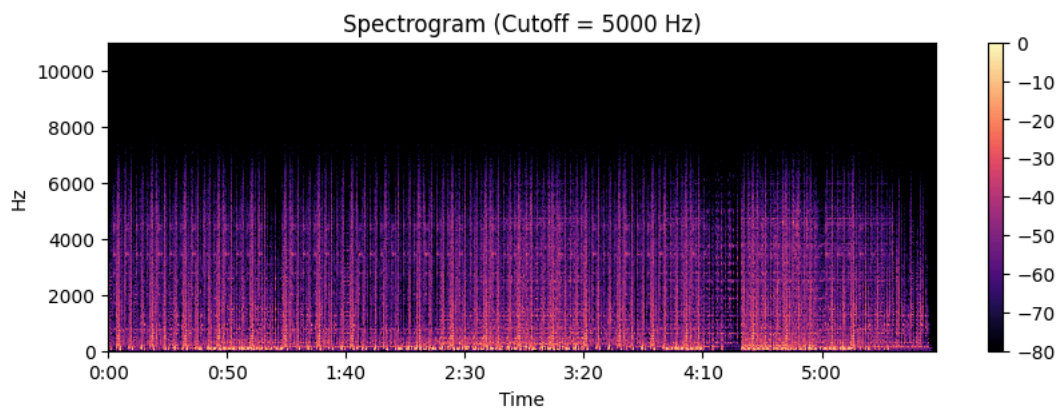


```
=====
```

LOW-PASS FILTER CUTOFF = 5000 Hz

```
=====
```

0:00 / 5:48



Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

